

深度学习的统计学习框架

第 2 讲：损失、似然与正则化

范一苇

北京理工大学

学习目标

本讲要求：

- ① 区分条件分布、似然函数与损失函数；
- ② 推导平方损失与交叉熵的概率来源；
- ③ 解释交叉熵的名称与使用理由；
- ④ 把样本学习写成经验风险最小化。

上节回顾

第 1 讲给出了学习问题的两个要素：

$$\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n, \quad \mathcal{F} = \{f_\theta : \theta \in \Theta\}.$$

数据 提供已观测的输入与输出；

模型空间 规定允许考虑的候选模型；

待解决问题 怎样比较候选模型，并从中选出一个模型？

本讲主线

损失函数 $\longrightarrow$ 经验风险.

损失函数

定义 1 (损失函数)

设真实输出为 $y \in \mathcal{Y}$, 模型行动或预测为 $a \in \mathcal{A}$ 。损失函数是映射

$$L : \mathcal{Y} \times \mathcal{A} \longrightarrow [0, \infty),$$

其中 $L(y, a)$ 衡量预测 a 与真实输出 y 的偏离。

- 回归中, a 可以是一个实数预测;
- 分类中, a 可以是类别标签或概率向量;

说明

损失不是由模型空间自动决定的; 同一模型空间可配合不同损失。

条件分布

对监督学习，模型常直接描述

$$p_{\theta}(y \mid x),$$

即给定输入 $X = x$ 时，输出 Y 的条件分布。

任务	条件分布	模型输出
实值回归	正态分布等	均值、方差
二分类	Bernoulli 分布	正类概率
多分类	类别分布	各类别概率
计数回归	Poisson 分布等	条件均值

参数 θ 决定分布的位置、形状或类别概率。

概率与密度

$p_{\theta}(y | x)$ 可表示概率质量函数或概率密度函数。

离散输出 $p_{\theta}(y | x) = P_{\theta}(Y = y | X = x)$, 可直接解释为概率;

连续输出 $p_{\theta}(y | x)$ 是密度, 区间概率由积分得到:

$$P_{\theta}(a \leq Y \leq b | X = x) = \int_a^b p_{\theta}(y | x) dy.$$

说明

连续变量在单点上的概率为零。似然使用观测点处的密度值, 而不是“该点发生的概率”。

似然函数

观测到 (x, y) 后，将数据固定，把

$$\mathcal{L}(\theta; x, y) = p_{\theta}(y \mid x)$$

视为参数 θ 的函数，称为似然函数。

表达	固定对象	变化对象
分布 $p_{\theta}(y \mid x)$	参数 θ 、输入 x	可能输出 y
似然 $\mathcal{L}(\theta; x, y)$	观测数据 (x, y)	参数 θ

$$\hat{\theta}_{\text{MLE}} \in \arg \max_{\theta \in \Theta} \mathcal{L}(\theta; x, y).$$

说明

最大似然选择最支持观测数据的参数。似然不是参数的概率分布。

样本似然

对独立样本

$$\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n,$$

条件似然为

$$\mathcal{L}_n(\theta) = \prod_{i=1}^n p_{\theta}(y_i \mid x_i).$$

- 每一项评价模型对一个观测输出的支持程度；
- 独立性使联合条件密度分解为乘积；
- 任一样本的似然很小，乘积都会明显减小。

最大似然估计为

$$\hat{\theta}_{\text{MLE}} \in \arg \max_{\theta \in \Theta} \prod_{i=1}^n p_{\theta}(y_i \mid x_i).$$

对数似然

对数函数严格递增，因此最大化似然等价于最大化对数似然：

$$\ell_n(\theta) = \log \mathcal{L}_n(\theta) = \sum_{i=1}^n \log p_{\theta}(y_i \mid x_i).$$

取对数有两个作用：

- ① 将乘积转化为求和，便于求导与分批计算；
- ② 避免大量小概率相乘造成数值下溢。

$$\arg \max_{\theta} \mathcal{L}_n(\theta) = \arg \max_{\theta} \ell_n(\theta).$$

负对数似然

优化通常写为最小化问题，因此定义单样本负对数似然损失

$$L_{\theta}(x, y) = -\log p_{\theta}(y \mid x).$$

样本平均负对数似然为

$$\frac{1}{n} \sum_{i=1}^n L_{\theta}(x_i, y_i) = -\frac{1}{n} \ell_n(\theta).$$

因而

$$\arg \max_{\theta} \mathcal{L}_n(\theta) = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n [-\log p_{\theta}(y_i \mid x_i)].$$

三者关系

概念	数学形式	作用
条件分布	$p_{\theta}(y \mid x)$	描述输出的随机规律
似然	$\mathcal{L}_n(\theta) = \prod_i p_{\theta}(y_i \mid x_i)$	用观测数据比较参数
单样本损失	$L_{\theta}(x_i, y_i) = -\log p_{\theta}(y_i \mid x_i)$	衡量单个观测的偏离
样本目标	$\frac{1}{n} \sum_i L_{\theta}(x_i, y_i)$	汇总样本并用于优化

分布假设不同，导出的负对数似然损失也不同。

正态回归

设模型给出条件均值 $f_{\theta}(x)$ ，并假定

$$Y \mid X = x \sim N(f_{\theta}(x), \sigma^2).$$

条件密度为

$$p_{\theta}(y \mid x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left\{ -\frac{(y - f_{\theta}(x))^2}{2\sigma^2} \right\}.$$

该假设包含两层含义：

- $f_{\theta}(x)$ 描述给定输入后的中心位置；
- σ^2 描述观测围绕中心的波动。

平方损失

对正态条件密度取负对数：

$$-\log p_{\theta}(y \mid x) = \frac{1}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} (y - f_{\theta}(x))^2.$$

若 σ^2 固定，则第一项与 θ 无关，第二项只是平方误差的正常数倍。因此

$$\arg \min_{\theta} \sum_{i=1}^n [-\log p_{\theta}(y_i \mid x_i)] = \arg \min_{\theta} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2.$$

命题 1

固定方差的正态回归中，最大似然估计等价于最小化平方损失。

回归例子

设观测值 $y = 3$ ，且 $\sigma^2 = 1$ 。

预测均值 μ	误差 $y - \mu$	平方损失	负对数似然
2.0	1.0	1.00	$\frac{1}{2} \log(2\pi) + 0.50 \approx 1.419$
2.8	0.2	0.04	$\frac{1}{2} \log(2\pi) + 0.02 \approx 0.939$
4.5	-1.5	2.25	$\frac{1}{2} \log(2\pi) + 1.125 \approx 2.044$

三种预测的排序在两种准则下完全一致，因为负对数似然是平方损失的线性变换。

Bernoulli 模型

二分类标签取 $Y \in \{0, 1\}$ 。模型输出

$$p_{\theta}(x) = P_{\theta}(Y = 1 \mid X = x).$$

Bernoulli 条件概率质量函数可统一写为

$$p_{\theta}(y \mid x) = p_{\theta}(x)^y \{1 - p_{\theta}(x)\}^{1-y}.$$

观测标签	代入后的概率	含义
$y = 1$	$p_{\theta}(x)$	模型给真实正类的概率
$y = 0$	$1 - p_{\theta}(x)$	模型给真实负类的概率

二分类交叉熵

对 Bernoulli 模型取负对数：

$$L\{y, p_{\theta}(x)\} = -y \log p_{\theta}(x) - (1 - y) \log\{1 - p_{\theta}(x)\}.$$

分情况看：

$$L = \begin{cases} -\log p_{\theta}(x), & y = 1, \\ -\log\{1 - p_{\theta}(x)\}, & y = 0. \end{cases}$$

损失只考察模型分配给**真实类别**的概率。该概率越大，损失越小。

说明

二分类交叉熵就是 Bernoulli 条件模型的负对数似然。

名称来源

设真实分布为 $q = (q_1, \dots, q_K)$ ，模型预测分布为 $p = (p_1, \dots, p_K)$ 。

$$\text{熵: } H(q) = \mathbb{E}_{Y \sim q}[-\log q_Y] = - \sum_{c=1}^K q_c \log q_c,$$

$$\text{交叉熵: } H(q, p) = \mathbb{E}_{Y \sim q}[-\log p_Y] = - \sum_{c=1}^K q_c \log p_c.$$

为什么称为“交叉”

平均时使用真实分布 q 作为权重，却使用预测分布 p 的 $-\log p_c$ 衡量信息量；一个表达式同时涉及两个分布。

名称来源

交叉熵:
$$H(q, p) = \mathbb{E}_{Y \sim q}[-\log p_Y] = - \sum_{c=1}^K q_c \log p_c.$$

两种标签形式:

- q 为软标签时, 所有类别均可能对损失产生贡献;
- q 为 one-hot 标签时, 若真实类别为 y , 则

$$q_y = 1, \quad q_c = 0 \ (c \neq y), \quad \implies \quad H(q, p) = -\log p_y.$$

为何采用

交叉熵满足分解

$$H(q, p) = H(q) + D_{\text{KL}}(q \| p) \geq H(q),$$

等号成立当且仅当 $p = q$ 。真实分布 q 固定时，最小化交叉熵等价于使预测分布逼近 q 。

- **统计含义：** 期望交叉熵在真实概率处达到最小；
- **似然含义：** 样本平均交叉熵就是分类模型的平均负对数似然；
- **预测含义：** 既判断类别，也评价预测置信度；
- **优化含义：** 相比 0-1 损失，交叉熵提供连续的误差信号。

因此交叉熵同时连接概率建模、参数估计与数值优化。

二分类样本

设三个样本的标签与正类预测概率为

$$y = (1, 0, 1), \quad p = (0.8, 0.3, 0.6).$$

模型对实际观测标签赋予的概率分别为

$$0.8, \quad 1 - 0.3 = 0.7, \quad 0.6.$$

因而

$$\begin{aligned} \mathcal{L}_3(\theta) &= 0.8 \times 0.7 \times 0.6 = 0.336, \\ -\log \mathcal{L}_3(\theta) &\approx 1.090, \quad -\frac{1}{3} \log \mathcal{L}_3(\theta) \approx 0.363. \end{aligned}$$

说明

样本平均负对数似然就是这三个样本的平均交叉熵。

置信度惩罚

当真实标签为 $y = 1$ 时，损失为 $-\log p$ 。

预测概率 p	判断	损失 $-\log p$
0.90	正确且有把握	0.105
0.60	正确但把握有限	0.511
0.10	错误且很有把握	2.303
0.01	严重错误	4.605

交叉熵不仅关心分类是否正确，还评价概率是否合理；对高置信度错误的惩罚尤其大。

原始分数

神经网络通常先输出未经归一化的实数分数：

$$z = \boldsymbol{w}^T \boldsymbol{x} + b, \quad z \in \mathbb{R}.$$

这个分数称为 **logit**。

- logit 可以取任意实数；
- logit 越大，模型越倾向于预测正类；
- logit 不是概率，不能直接解释为置信度。

logit	模型倾向	是否为概率
$z < 0$	倾向于负类	否
$z = 0$	两类倾向相同	否
$z > 0$	倾向于正类	否

二分类概率

对二分类问题，使用 Sigmoid 函数将 logit 转换为概率：

$$p = P(Y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + \exp(-z)}.$$

logit z	概率 $\sigma(z)$	模型判断
-2	0.119	倾向于负类
0	0.500	两类倾向相同
2	0.881	倾向于正类

Sigmoid 的反函数为

$$z = \log \frac{p}{1 - p},$$

因此 logit 也称为对数优势。

多分类模型

设 $Y \in \{1, \dots, K\}$ 。模型输出概率向量

$$p_{\theta}(x) = (p_{\theta,1}(x), \dots, p_{\theta,K}(x)), \quad \sum_{c=1}^K p_{\theta,c}(x) = 1.$$

类别分布可写为

$$p_{\theta}(y \mid x) = \prod_{c=1}^K p_{\theta,c}(x)^{I(y=c)}.$$

指示函数 $I(y = c)$ 只在真实类别处取 1，其余类别取 0。

多分类交叉熵

多分类负对数似然（交叉熵）为

$$L\{y, p_{\theta}(x)\} = - \sum_{c=1}^K I(y = c) \log p_{\theta,c}(x) = -\log p_{\theta,y}(x).$$

例 2

若真实类别为第 2 类，预测概率为

$$p = (0.1, 0.7, 0.2),$$

则损失为 $-\log 0.7 \approx 0.357$ 。若真实类别概率降至 0.05，损失增至 $-\log 0.05 \approx 2.996$ 。

Softmax

对 K 分类问题，模型为每个类别输出一个 logit:

$$\mathbf{z} = (z_1, z_2, \dots, z_K).$$

Softmax 将这些分数转换为类别概率:

$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}, \quad k = 1, \dots, K.$$

转换后的结果满足

$$p_k \geq 0, \quad \sum_{k=1}^K p_k = 1.$$

Softmax 比较不同类别的分数，并将它们归一化为一个概率分布。

计算示例

假设三个类别的 logits 为

$$z = (2, 1, 0).$$

第一步，对各分量取指数：

$$\exp(z) = (e^2, e^1, e^0) \approx (7.389, 2.718, 1).$$

第二步，计算归一化结果：

$$\text{softmax}(z) = \frac{(7.389, 2.718, 1)}{7.389 + 2.718 + 1} \approx (0.665, 0.245, 0.090).$$

第一个类别的 logit 最大，因此其预测概率也最大。

回归目标

并非所有损失都必须从似然导出。任务目标或稳健性要求也可直接决定损失。

损失	数学形式	主要特点
平方损失	$(y - a)^2$	强调较大误差，对异常值敏感
绝对损失	$ y - a $	对异常值更稳健
Huber 损失	小误差二次、大误差线性	兼顾平滑性与稳健性
分位数损失	$(y - a)[\tau - I(y < a)]$	学习条件分位数

损失选择同时反映预测目标与对误差分布的假设。

分类目标

交叉熵 对概率预测进行评价，来源于 Bernoulli 或类别分布的负对数似然。

Hinge 损失 对 $y \in \{-1, +1\}$ 与判别分数 a ，定义

$$L_{\text{hinge}}(y, a) = \max\{0, 1 - ya\}.$$

当 $ya \geq 1$ 时损失为零；否则按间隔不足程度增加。

0-1 损失 分类错误时取 1，正确时取 0。它直接对应错误率，但不便于梯度优化。

概率预测通常使用交叉熵；只需判别边界时，也可采用基于间隔的准则。

函数形式

损失函数接收预测与目标，并返回单样本损失或聚合损失：

```
def squared_loss(pred, target, reduction="mean"):
    # Keep one loss for every prediction
    loss = (pred - target).square()

    if reduction == "none":
        return loss          # no aggregation
    if reduction == "sum":
        return loss.sum()    # total loss
    if reduction == "mean":
        return loss.mean()   # average loss
    raise ValueError("unknown reduction")
```

先计算逐样本损失，再由 `reduction` 决定聚合方式。保留逐样本结果，便于检查异常样本或加入权重。

聚合输出

比较 none、sum 与 mean:

```
import torch
pred = torch.tensor([2.0, 3.0, 5.0])
target = torch.tensor([1.0, 4.0, 4.0])
print(squared_loss(pred, target, "none"))
print(squared_loss(pred, target, "sum").item())
print(squared_loss(pred, target, "mean").item())
```

输出

```
tensor([1., 1., 1.])
3.0
1.0
```

三个样本的平方损失均为 1；sum 随批量大小变化，mean 更便于比较不同批量的训练结果。

平方损失

内置损失模块可以直接参与自动微分：

```
import torch
from torch import nn

# pred needs gradients; target does not
pred = torch.tensor([2.0, 3.0, 5.0], requires_grad=True)
target = torch.tensor([1.0, 4.0, 4.0])

criterion = nn.MSELoss() # default: reduction="mean"
loss = criterion(pred, target)
manual = (pred - target).square().mean()
loss.backward()          # fill pred.grad
print(loss.item(), manual.item(), pred.grad)
```

`backward()` 从标量损失出发，将梯度写入 `pred.grad`。

接口参考： `torch.nn.MSELoss`。

平方输出

上一页程序输出为

```
1.0 1.0 tensor([ 0.6667, -0.6667,  0.6667])
```

- 内置损失与手工计算均为 1;
- 对批量平均平方损失, 梯度为

$$\frac{2}{3}(\text{pred} - \text{target});$$

- 梯度为正时应减小预测值, 为负时应增大预测值。

损失输入

二分类

`BCEWithLogitsLoss` 直接接收一个 logit:

$$z \longrightarrow \text{Sigmoid} \longrightarrow \text{二分类交叉熵}.$$

不需要提前对 z 使用 Sigmoid。

多分类

`CrossEntropyLoss` 直接接收 K 个 logits:

$$(z_1, \dots, z_K) \longrightarrow \text{Softmax 的对数形式} \longrightarrow \text{多分类交叉熵}.$$

不需要提前对 logits 使用 Softmax。

二分类损失

```
import torch
from torch import nn

# Raw scores and floating-point targets
logits = torch.tensor([1.2, -0.4, 0.8], requires_grad=True)
target = torch.tensor([1.0, 0.0, 1.0])

criterion = nn.BCEWithLogitsLoss(reduction="none")
loss_i = criterion(logits, target) # one loss per sample
loss = loss_i.mean()
loss.backward()
```

不要重复 Sigmoid

直接输入 logits；该模块内部完成 Sigmoid 与交叉熵，数值更稳定。

二分类输出

将 logits 转成概率，并查看逐样本损失：

```
prob = torch.sigmoid(logits.detach())
print(prob.round(decimals=4))
print(loss_i.detach().round(decimals=4))
print(round(loss.item(), 4))
```

```
tensor([0.7685, 0.4013, 0.6900])
tensor([0.2633, 0.5130, 0.3711])
0.3825
```

第一行是正类概率，第二行是三个样本的交叉熵，最后一行是批量平均损失。目标必须与 logits 形状相同，并使用 $[0, 1]$ 内的浮点数。

多分类损失

B 个样本、 K 个类别时, logits 形状为 (B, K) :

```
# Two samples, three class scores per sample
logits = torch.tensor([[2.0, 0.5, -1.0],
                       [0.1, 0.2, 1.8]], requires_grad=True)
target = torch.tensor([0, 2], dtype=torch.long) # class index
criterion = nn.CrossEntropyLoss(reduction="none")
loss_i = criterion(logits, target)
loss = loss_i.mean()
loss.backward()
```

输入形状为 (B, K) ; 类别索引形状为 (B) , 取值为 $0, \dots, K - 1$ 。直接输入 logits, 不应预先调用 Softmax。

分类输出

接续上一页，查看 Softmax 概率与逐样本交叉熵：

```
prob = torch.softmax(logits.detach(), dim=1)

print("prob:", prob.round(decimals=4))
print("loss_i:", loss_i.detach().round(decimals=4))
print("mean:", round(loss.item(), 4))
```

```
prob: tensor([[0.7856, 0.1753, 0.0391],
              [0.1319, 0.1458, 0.7222]])
loss_i: tensor([0.2413, 0.3254])
mean: 0.2834
```

- 第一个样本的真实类别为 0，故损失为 $-\log(0.7856) = 0.2413$ ；
- 第二个样本的真实类别为 2，故损失为 $-\log(0.7222) = 0.3254$ ；
- 批量损失是两个逐样本损失的平均值。

自定义模块

需要样本权重时，可继承 `nn.Module`：

```
import torch
from torch import nn

class WeightedMSELoss(nn.Module):
    def forward(self, pred, target, weight):
        loss_i = weight * (pred - target).square()
        return loss_i.sum() / weight.sum() # weighted mean

pred = torch.tensor([2., 3., 5.], requires_grad=True)
target = torch.tensor([1., 4., 4.])
weight = torch.tensor([1.0, 1.0, 3.0])
loss = WeightedMSELoss()(pred, target, weight)
loss.backward()
```

`forward` 规定损失的前向计算；自动微分负责生成反向计算。

加权输出

查看加权平均损失与预测梯度：

```
print(round(loss.item(), 4), pred.grad)
```

```
1.0 tensor([ 0.4000, -0.4000,  1.2000])
```

- 三个样本的平方误差均为 1，规范化加权平均仍为 1；
- 第三个样本的权重为 3，其梯度绝对值也更大；
- 除以 `weight.sum()`，可避免权重总量任意改变损失尺度。

样本平均

单样本损失只评价一个观测：

$$L(y_i, f(x_i)).$$

为评价模型在整个训练集上的表现，对全部单样本损失取平均：

$$\frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)).$$

层次	回答的问题
单样本损失	模型在第 i 个样本上错了多少？
样本平均损失	模型在当前数据集上总体表现如何？

经验风险

定义 3 (经验风险)

对样本 $\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n$ 、候选函数 f 与损失 L ，经验风险定义为

$$R_{\text{emp}}(f; \mathcal{D}_n) = \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)).$$

- 经验风险由有限训练样本计算；
- 损失函数决定“误差”的含义；
- 模型空间决定可以在什么范围内搜索。

经验风险最小化

给定模型空间 $\mathcal{F}$ 与损失函数 L ，经验风险最小化为

$$\hat{f} \in \arg \min_{f \in \mathcal{F}} R_{\text{emp}}(f) = \arg \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)).$$

对参数化模型 $\mathcal{F} = \{f_{\theta} : \theta \in \Theta\}$:

$$\hat{\theta} \in \arg \min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n L(y_i, f_{\theta}(x_i)).$$

最大似然是经验风险最小化的特例，此时 $L = -\log p_{\theta}$ 。

最小二乘

在线性函数类中

$$f_{\theta}(x) = \beta_0 + \sum_{j=1}^p \beta_j x^{(j)},$$

取平方损失 $L(y, a) = (y - a)^2$, 经验风险为

$$R_{\text{emp}}(\theta) = \frac{1}{n} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_i^{(j)} \right)^2.$$

因此

$$\hat{\theta} \in \arg \min_{\theta} R_{\text{emp}}(\theta)$$

就是最小二乘估计。

训练风险的局限

经验风险只评价模型对训练样本的拟合程度。模型空间较大时，最小经验风险可能来自对随机波动的拟合。

现象	训练集表现	新样本表现
拟合不足	误差较大	误差通常也较大
适度拟合	误差较小	误差较小
过度拟合	误差很小	误差可能增大

说明

训练误差降低不保证未知数据上的误差降低。第 3 讲将用期望风险与泛化误差刻画这一差异。

小结

- ① 条件分布描述输出的随机规律；
- ② 代入观测数据后，条件分布形成参数的似然函数；
- ③ 负对数似然把最大化似然转为最小化损失；
- ④ 正态、Bernoulli 与类别分布分别导出平方损失和交叉熵；
- ⑤ 经验风险汇总单样本损失。

核心关系

$$p_{\theta}(y \mid x) \implies \mathcal{L}_n(\theta) \implies -\frac{1}{n} \log \mathcal{L}_n(\theta) \implies R_{\text{emp}}(\theta).$$

下一讲：为什么训练样本上得到的模型可以推广到新样本？